

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря
Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2

з дисципліни «Бази даних»

Виконав:

Машута Олександр
студент групи ІМ-051
номер у списку групи: 16

Перевірів:
Русінов В.В.

Київ 2026

1. Короткий опис схеми

Лабораторна робота №2 присвячена створенню структури бази даних відеохостинг-платформи **VideoHub** за допомогою мови визначення даних **DDL** (Data Definition Language). На основі ER-діаграмми, спроектованої в Lab 1, необхідно було перетворити концептуальну модель у реляційну схему PostgreSQL: створити таблиці, визначити типи стовпців, первинні та зовнішні ключі, унікальні обмеження, перевірочні правила `CHECK` та індекси.

Обраний технологічний стек — **PostgreSQL** як об'єктно-реляційна СКБД. Для ефективної роботи з email і username використано розширення `citext`, яке дозволяє порівнювати рядки без урахування регістру, уникаючи дублікатів на кшталт `User@mail.com` і `user@mail.com`.

Схема складається з п'яти таблиць:

- `user` — зареєстровані користувачі/канали;
- `video` — відеоролики, завантажені каналами;
- `comment` — коментарі користувачів до відео з підтримкою відповідей;
- `like` — вподобайки (лайки/дизлайки) до відео;
- `subscription` — підписки одного користувача на іншого (M:N само-посилання на `user`).

Зв'язки між сутностями реалізовано через зовнішні ключі з правилом `ON DELETE CASCADE`, що забезпечує цілісність даних: при видаленні батьківського запису дочірні записи видаляються автоматично, не залишаючи «сиріт».

2. SQL-код створення схеми

2.1. Підключення розширення `citext`

Для полів `username` та `email` у таблиці `user` використано тип `CITEXT`, який є регістронезалежним аналогом `VARCHAR`. Це важливо для автентифікації, щоб `TechWithDeny` і `techwithdeny` сприймалися як одна й та сама назва.

```
CREATE EXTENSION IF NOT EXISTS citext;
```



2.2. Таблиця user

Таблиця user зберігає інформацію про зареєстрованих користувачів. Вона є «батьківською» для всіх інших таблиць, оскільки відео, коментарі, вподобайки та підписки залежать від користувачів.

Атрибути та обмеження:

- user_id — автоінкрементний BIGSERIAL , первинний ключ;
- username — CITEXT(30) , унікальний, NOT NULL;
- email — CITEXT(255) , унікальний, NOT NULL;
- password_hash — VARCHAR(255) , може бути NULL (Google OAuth);
- google_id — VARCHAR(255) , унікальний, може бути NULL (парольна автентифікація);
- avatar_url , bio — TEXT , необов'язкові;
- is_channel — BOOLEAN NOT NULL DEFAULT FALSE ;
- created_at — TIMESTAMPTZ NOT NULL DEFAULT NOW() .

```
CREATE TABLE "user" (  
    user_id          BIGSERIAL  
                    CONSTRAINT pk_user PRIMARY KEY,  
  
    username         CITEXT(30)  
                    CONSTRAINT uq_user_username UNIQUE  
                    CONSTRAINT nn_user_username NOT NULL,  
  
    email            CITEXT(255)  
                    CONSTRAINT uq_user_email UNIQUE  
                    CONSTRAINT nn_user_email NOT NULL,  
  
    password_hash    VARCHAR(255),  
  
    google_id        VARCHAR(255)  
                    CONSTRAINT uq_user_google_id UNIQUE,  
  
    avatar_url       TEXT,  
  
    bio              TEXT,  
  
    is_channel       BOOLEAN  
                    CONSTRAINT nn_user_is_channel NOT NULL  
                    DEFAULT FALSE,  
  
    created_at       TIMESTAMPTZ  
                    CONSTRAINT nn_user_created_at NOT NULL  
                    DEFAULT NOW()  
);  
  
CREATE INDEX idx_user_is_channel ON "user" (is_channel);
```

Припущення: користувач може використовувати або пароль, або Google OAuth, або обидва способи. Схема дозволяє password_hash і google_id бути NULL окремо, але не обидва водночас — це бізнес-інваріант, який перевіряється на рівні застосунку.

2.3. Таблиця video

Таблиця video зберігає метадані відеороликів. Кожне відео належить рівно одному користувачеві-каналу.

Атрибути та обмеження:

- video_id — BIGSERIAL , первинний ключ;
- user_id — BIGINT NOT NULL , зовнішній ключ до user(user_id) з ON DELETE CASCADE ;
- title — VARCHAR(255) NOT NULL ;
- description , thumbnail_url — TEXT , необов'язкові;
- video_url — TEXT NOT NULL ;
- duration_seconds — INTEGER NOT NULL CHECK (duration_seconds > 0) ;
- views — INTEGER NOT NULL DEFAULT 0 CHECK (views >= 0) ;
- is_public — BOOLEAN NOT NULL DEFAULT TRUE ;
- created_at — TIMESTAMPTZ NOT NULL DEFAULT NOW() .

```
CREATE TABLE video (  
    video_id          BIGSERIAL  
                      CONSTRAINT pk_video PRIMARY KEY,  
  
    user_id           BIGINT  
                      CONSTRAINT nn_video_user_id NOT NULL  
                      CONSTRAINT fk_video_user FOREIGN KEY REFERENCES "user" (user_id)  
                      ON DELETE CASCADE,  
  
    title             VARCHAR(255)  
                      CONSTRAINT nn_video_title NOT NULL,  
  
    description        TEXT,  
  
    video_url         TEXT  
                      CONSTRAINT nn_video_url NOT NULL,  
  
    thumbnail_url     TEXT,  
  
    duration_seconds  INTEGER  
                      CONSTRAINT nn_video_duration NOT NULL  
                      CONSTRAINT chk_video_duration CHECK (duration_seconds > 0),  
  
    views             INTEGER  
                      CONSTRAINT nn_video_views NOT NULL  
                      CONSTRAINT chk_video_views CHECK (views >= 0)  
                      DEFAULT 0,  
  
    is_public         BOOLEAN  
                      CONSTRAINT nn_video_is_public NOT NULL  
                      DEFAULT TRUE,  
  
    created_at        TIMESTAMPTZ  
                      CONSTRAINT nn_video_created_at NOT NULL  
                      DEFAULT NOW())
```

```
);
```

```
CREATE INDEX idx_video_user ON video (user_id);  
CREATE INDEX idx_video_public ON video (is_public) WHERE is_public = TRUE;
```

Правило `ON DELETE CASCADE` на `user_id` означає: видалення каналу спричиняє видалення всіх його відео. Це логічно, оскільки відео без власника не можуть існувати.

2.4. Таблиця `comment`

Таблиця `comment` зберігає коментарі до відео та підтримує вкладеність через self-referencing FK `parent_comment_id`.

Атрибути та обмеження:

- `comment_id` — `BIGSERIAL`, первинний ключ;
- `user_id` — `BIGINT NOT NULL`, FK до `user(user_id)`, `ON DELETE CASCADE`;
- `video_id` — `BIGINT NOT NULL`, FK до `video(video_id)`, `ON DELETE CASCADE`;
- `parent_comment_id` — `BIGINT`, self-referencing FK до `comment(comment_id)`, `ON DELETE CASCADE`;
- `text` — `TEXT NOT NULL`;
- `created_at` — `TIMESTAMPZ NOT NULL DEFAULT NOW()`.

```
CREATE TABLE comment (  
    comment_id          BIGSERIAL  
                        CONSTRAINT pk_comment PRIMARY KEY,  
  
    user_id             BIGINT  
                        CONSTRAINT nn_comment_user_id NOT NULL  
                        CONSTRAINT fk_comment_user FOREIGN KEY REFERENCES "user" (user_id)  
                        ON DELETE CASCADE,  
  
    video_id            BIGINT  
                        CONSTRAINT nn_comment_video_id NOT NULL  
                        CONSTRAINT fk_comment_video FOREIGN KEY REFERENCES video (video_id)  
                        ON DELETE CASCADE,  
  
    parent_comment_id   BIGINT  
                        CONSTRAINT fk_comment_parent FOREIGN KEY REFERENCES comment  
(comment_id)  
                        ON DELETE CASCADE,  
  
    text                TEXT  
                        CONSTRAINT nn_comment_text NOT NULL,  
  
    created_at          TIMESTAMPZ  
                        CONSTRAINT nn_comment_created_at NOT NULL  
                        DEFAULT NOW()  
);
```

```
CREATE INDEX idx_comment_video ON comment (video_id);
CREATE INDEX idx_comment_parent ON comment (parent_comment_id);
```

2.5. Таблиця like

Таблиця like зберігає реакції користувачів на відео: лайк (is_like = TRUE) або дизлайк (is_like = FALSE). Обмеження UNIQUE(user_id, video_id) гарантує, що один користувач може поставити лише одну оцінку на відео.

Атрибути та обмеження:

- like_id — BIGSERIAL , первинний ключ;
- user_id , video_id — BIGINT NOT NULL , FK до user і video з ON DELETE CASCADE ;
- is_like — BOOLEAN NOT NULL ;
- created_at — TIMESTAMPTZ NOT NULL DEFAULT NOW() ;
- UNIQUE(user_id, video_id) .

```
CREATE TABLE "like" (
    like_id          BIGSERIAL
                    CONSTRAINT pk_like PRIMARY KEY,

    user_id          BIGINT
                    CONSTRAINT nn_like_user_id NOT NULL
                    CONSTRAINT fk_like_user FOREIGN KEY REFERENCES "user" (user_id)
                    ON DELETE CASCADE,

    video_id         BIGINT
                    CONSTRAINT nn_like_video_id NOT NULL
                    CONSTRAINT fk_like_video FOREIGN KEY REFERENCES video (video_id)
                    ON DELETE CASCADE,

    is_like          BOOLEAN
                    CONSTRAINT nn_like_is_like NOT NULL,

    created_at       TIMESTAMPTZ
                    CONSTRAINT nn_like_created_at NOT NULL
                    DEFAULT NOW(),

    CONSTRAINT uq_like_user_video UNIQUE (user_id, video_id)
);

CREATE INDEX idx_like_video ON "like" (video_id);
```

2.6. Таблиця subscription

Таблиця subscription реалізує M:N-зв'язок між користувачами (self-referencing): підписник і канал — це однакова таблиця user . Складений первинний ключ (subscriber_id, channel_id) унеможливорює дублювання підписки.

Атрибути та обмеження:

- subscriber_id , channel_id — BIGINT NOT NULL , FK до user(user_id) , ON DELETE CASCADE ;


```

    avatar_url    TEXT,
    bio           TEXT,
    is_channel    BOOLEAN
                CONSTRAINT nn_user_is_channel NOT NULL
                DEFAULT FALSE,
    created_at    TIMESTAMPTZ
                CONSTRAINT nn_user_created_at NOT NULL
                DEFAULT NOW()
);

CREATE INDEX idx_user_is_channel ON "user" (is_channel);

CREATE TABLE video (
    video_id      BIGSERIAL
                CONSTRAINT pk_video PRIMARY KEY,
    user_id       BIGINT
                CONSTRAINT nn_video_user_id NOT NULL
                CONSTRAINT fk_video_user FOREIGN KEY REFERENCES "user" (user_id)
                ON DELETE CASCADE,
    title         VARCHAR(255)
                CONSTRAINT nn_video_title NOT NULL,
    description    TEXT,
    video_url     TEXT
                CONSTRAINT nn_video_url NOT NULL,
    thumbnail_url TEXT,
    duration_seconds INTEGER
                CONSTRAINT nn_video_duration NOT NULL
                CONSTRAINT chk_video_duration CHECK (duration_seconds > 0),
    views         INTEGER
                CONSTRAINT nn_video_views NOT NULL
                CONSTRAINT chk_video_views CHECK (views >= 0)
                DEFAULT 0,
    is_public     BOOLEAN
                CONSTRAINT nn_video_is_public NOT NULL
                DEFAULT TRUE,
    created_at    TIMESTAMPTZ
                CONSTRAINT nn_video_created_at NOT NULL
                DEFAULT NOW()
);

CREATE INDEX idx_video_user ON video (user_id);
CREATE INDEX idx_video_public ON video (is_public) WHERE is_public = TRUE;

CREATE TABLE comment (
    comment_id    BIGSERIAL
                CONSTRAINT pk_comment PRIMARY KEY,
    user_id       BIGINT
                CONSTRAINT nn_comment_user_id NOT NULL
                CONSTRAINT fk_comment_user FOREIGN KEY REFERENCES "user" (user_id)
                ON DELETE CASCADE,
    video_id      BIGINT
                CONSTRAINT nn_comment_video_id NOT NULL

```

```

        CONSTRAINT fk_comment_video FOREIGN KEY REFERENCES video (video_id)
        ON DELETE CASCADE,
parent_comment_id BIGINT
        CONSTRAINT fk_comment_parent FOREIGN KEY REFERENCES comment
(comment_id)
        ON DELETE CASCADE,
text TEXT
        CONSTRAINT nn_comment_text NOT NULL,
created_at TIMESTAMPTZ
        CONSTRAINT nn_comment_created_at NOT NULL
        DEFAULT NOW()
);

CREATE INDEX idx_comment_video ON comment (video_id);
CREATE INDEX idx_comment_parent ON comment (parent_comment_id);

CREATE TABLE "like" (
    like_id BIGSERIAL
        CONSTRAINT pk_like PRIMARY KEY,
    user_id BIGINT
        CONSTRAINT nn_like_user_id NOT NULL
        CONSTRAINT fk_like_user FOREIGN KEY REFERENCES "user" (user_id)
        ON DELETE CASCADE,
    video_id BIGINT
        CONSTRAINT nn_like_video_id NOT NULL
        CONSTRAINT fk_like_video FOREIGN KEY REFERENCES video (video_id)
        ON DELETE CASCADE,
    is_like BOOLEAN
        CONSTRAINT nn_like_is_like NOT NULL,
    created_at TIMESTAMPTZ
        CONSTRAINT nn_like_created_at NOT NULL
        DEFAULT NOW(),
    CONSTRAINT uq_like_user_video UNIQUE (user_id, video_id)
);

CREATE INDEX idx_like_video ON "like" (video_id);

CREATE TABLE subscription (
    subscriber_id BIGINT
        CONSTRAINT nn_sub_subscriber NOT NULL
        CONSTRAINT fk_sub_subscriber FOREIGN KEY REFERENCES "user" (user_id)
        ON DELETE CASCADE,
    channel_id BIGINT
        CONSTRAINT nn_sub_channel NOT NULL
        CONSTRAINT fk_sub_channel FOREIGN KEY REFERENCES "user" (user_id)
        ON DELETE CASCADE,
    created_at TIMESTAMPTZ
        CONSTRAINT nn_sub_created_at NOT NULL
        DEFAULT NOW(),
    CONSTRAINT pk_subscription PRIMARY KEY (subscriber_id, channel_id),
    CONSTRAINT chk_sub_no_self CHECK (subscriber_id <> channel_id)
);

```

```
CREATE INDEX idx_sub_channel ON subscription (channel_id);
```



3. SQL-код заповнення даними

3.1. Порядок вставок

Вставки виконуються в такому порядку:

1. **user** — батьківська таблиця без FK;
2. **video** — посилається на **user** ;
3. **comment** — посилається на **user** і **video** ;
4. **like** — посилається на **user** і **video** ;
5. **subscription** — обидва стовпці посилаються на **user** .

3.2. Тестові дані

Файл `inserts.sql` містить реалістичні тестові записи, які відповідають бізнес-логіці платформи: є канали, звичайні глядачі, публічні та приватні відео, коментарі-відповіді, лайки, дизлайки та підписки.

```
INSERT INTO "user" (user_id, username, email, password_hash, google_id, avatar_url, bio, is_channel)
VALUES
(1, 'TechWithDeny', 'techwithdeny@videohub.io',
 '$2b$12$abcdefgh1234567890abcdefghijk1', NULL,
 'https://cdn.videohub.io/avatars/techwithdeny.jpg',
 'Огляди новітніх технологій, tutorіали з програмування та код-рев'ю.', TRUE),

(2, 'MusicVibes', 'vibes@musicvibes.com',
 '$2b$12$xyz9876543210qwertyuiopasdfghj', NULL,
 'https://cdn.videohub.io/avatars/musicvibes.png',
 'Кавери, оригінальні треки та живі виступи. Підписуйся!', TRUE),

(3, 'ScienceHubUA', 'science@sciencehub.ua',
 '$2b$12$mnbvcxz0987654321lkjhgfdsap', NULL,
 'https://cdn.videohub.io/avatars/sciencehub.png',
 'Цікаві факти про науку, космос, біологію та фізику простими словами.', TRUE),

(4, 'Olena_Guest', 'olena.guest@example.com',
 '$2b$12$qwerty1234567890asdfghjklzxcv', NULL, NULL, NULL, FALSE),

(5, 'AndriyViewer', 'andriy.viewer@example.com',
 '$2b$12$zxcvbnm0987654321poiuytrewqlk', NULL, NULL,
 'Люблю дивитися документалістки та технологічні відео.', FALSE),

(6, 'GoogleUserMax', 'max.smith@gmail.com',
 NULL, '109283746501928374650',
 'https://lh3.googleusercontent.com/a/maxsmith.jpg',
 'Увійшов через Google. Пароль не встановлено.', FALSE);
```

```
INSERT INTO video (video_id, user_id, title, description, video_url, thumbnail_url,
duration_seconds, views, is_public)
```

```
VALUES
```

```
(1, 1, 'PostgreSQL для початківців: повний туторіал 2026',
'У цьому відео розбираємо основи PostgreSQL...',
'https://cdn.videohub.io/videos/pg_tutorial_2026.mp4',
'https://cdn.videohub.io/thumbs/pg_tutorial.jpg', 1845, 24350, TRUE),

(2, 1, 'Docker + PostgreSQL: налаштування за 10 хвилин',
'Швидкий старт: піднімаємо PostgreSQL у Docker...',
'https://cdn.videohub.io/videos/docker_pg.mp4',
'https://cdn.videohub.io/thumbs/docker_pg.jpg', 632, 8920, TRUE),

(3, 2, 'Кавер "Місто весна" – живий виступ',
'Живий виступ на фестивалі "Музика незалежності"...',
'https://cdn.videohub.io/videos/musicvibes_live.mp4',
'https://cdn.videohub.io/thumbs/musicvibes_live.jpg', 278, 15420, TRUE),

(4, 3, 'Чому небо синє? Фізика атмосфери за 7 хвилин',
'Розповідь про розсіювання Релея...',
'https://cdn.videohub.io/videos/why_sky_blue.mp4',
'https://cdn.videohub.io/thumbs/why_sky_blue.jpg', 442, 51200, TRUE),

(5, 3, 'Як працюють чорні діри? Пояснення без формул',
'Наукове, але доступне пояснення чорних дір...',
'https://cdn.videohub.io/videos/black_holes.mp4',
'https://cdn.videohub.io/thumbs/black_holes.jpg', 915, 102300, TRUE),

(6, 1, 'Чернетка: новий огляд Rust',
'Незавершений огляд мови Rust. Не публікувати!',
'https://cdn.videohub.io/videos/rust_draft.mp4',
NULL, 1200, 0, FALSE);
```

```
INSERT INTO comment (comment_id, user_id, video_id, parent_comment_id, text)
```

```
VALUES
```

```
(1, 4, 1, NULL, 'Чудовий туторіал! Дуже зрозуміло пояснюєте JOIN-и. Дякую!'),
(2, 5, 1, NULL, 'На 12:34 є неточність – INNER JOIN не повертає рядки без
відповідності...'),
(3, 1, 1, 2, 'Дякую, що помітили! Виправлю в описі під відео...'),
(4, 4, 4, NULL, 'Нарешті зрозуміла, чому небо синє! Дякую за просте пояснення.'),
(5, 5, 5, NULL, 'Це відео просто неймовірне. Горизонт подій – одна з найцікавіших
концепцій у фізиці!'),
(6, 6, 3, NULL, 'Дуже класний кавер! Коли буде новий трек?'),
(7, 3, 5, 5, 'Дякую за підтримку! Наступне відео – про квантову запутаність,
готуйтеся!');
```

```
INSERT INTO "like" (like_id, user_id, video_id, is_like)
```

```
VALUES
```

```
(1, 4, 1, TRUE),
(2, 5, 1, TRUE),
(3, 6, 1, TRUE),
```

```
(4, 6, 2, FALSE),
(5, 4, 4, TRUE),
(6, 5, 4, TRUE),
(7, 5, 5, TRUE),
(8, 4, 3, TRUE),
(9, 6, 5, FALSE);
```

```
INSERT INTO subscription (subscriber_id, channel_id)
VALUES
(4, 1),
(4, 3),
(5, 1),
(5, 2),
(6, 2),
(3, 1),
(1, 2);
```



4. Перевірка результатів

4.1. SELECT * для кожної таблиці

Після вставок виконуємо перевірочні запити `SELECT *` для переконання, що дані записано коректно.

```
SELECT user_id, username, email, is_channel, google_id FROM "user";
SELECT video_id, user_id, title, duration_seconds, views, is_public FROM video;
SELECT comment_id, user_id, video_id, parent_comment_id, LEFT(text, 60) AS text_preview FROM
comment;
SELECT like_id, user_id, video_id, is_like FROM "like";
SELECT subscriber_id, channel_id FROM subscription;
```



4.2. Демонстрація роботи обмежень

Файл `inserts.sql` містить закоментовані приклади порушень цілісності. Розглянемо декілька ключових сценаріїв:

1. Порушення UNIQUE на `username` :

```
INSERT INTO "user" (user_id, username, email, password_hash, is_channel)
VALUES (10, 'TechWithDeny', 'duplicate@example.com', 'hash', FALSE);
```

Очікувана помилка:

```
ERROR: duplicate key value violates unique constraint "uq_user_username"
```

2. **Порушення UNIQUE на email :**

```
INSERT INTO "user" (user_id, username, email, password_hash, is_channel)
VALUES (11, 'UniqueUser', 'techwithdeny@videohub.io', 'hash', FALSE);
```

3. **Порушення унікальної пари (user_id, video_id) у like :**

```
INSERT INTO "like" (user_id, video_id, is_like)
VALUES (4, 1, FALSE);
```

4. **Порушення CHECK subscriber_id <> channel_id :**

```
INSERT INTO subscription (subscriber_id, channel_id)
VALUES (4, 4);
```

5. **Порушення FK:**

```
INSERT INTO video (user_id, title, video_url, duration_seconds)
VALUES (999, 'Неможливе відео', 'https://example.com/v.mp4', 300);
```

6. **Порушення CHECK duration_seconds > 0 :**

```
INSERT INTO video (user_id, title, video_url, duration_seconds)
VALUES (1, 'Нульова тривалість', 'https://example.com/v2.mp4', 0);
```



5. Опис обмежень цілісності

5.1. Primary Key (PK)

Кожна таблиця має первинний ключ, який унікально ідентифікує рядок:

Таблиця	Первинний ключ
user	user_id (BIGSERIAL)
video	video_id (BIGSERIAL)
comment	comment_id (BIGSERIAL)
like	like_id (BIGSERIAL)
subscription	(subscriber_id, channel_id) — складений PK

5.2. Foreign Key (FK)

Зовнішні ключі реалізують зв'язки між сутностями та забезпечують референційну цілісність:

FK	Від таблиці	До таблиці	ON DELETE
fk_video_user	video.user_id	user.user_id	CASCADE
fk_comment_user	comment.user_id	user.user_id	CASCADE
fk_comment_video	comment.video_id	video.video_id	CASCADE
fk_comment_parent	comment.parent_comment_id	comment.comment_id	CASCADE
fk_like_user	like.user_id	user.user_id	CASCADE
fk_like_video	like.video_id	video.video_id	CASCADE
fk_sub_subscriber	subscription.subscriber_id	user.user_id	CASCADE
fk_sub_channel	subscription.channel_id	user.user_id	CASCADE

5.3. UNIQUE

- `uq_user_username` — унікальний нікнейм;
- `uq_user_email` — унікальна електронна пошта;
- `uq_user_google_id` — унікальний Google ID;
- `uq_like_user_video` — один користувач — одна реакція на відео.

5.4. CHECK

- `chk_video_duration : duration_seconds > 0` — тривалість відео додатна;
- `chk_video_views : views >= 0` — кількість переглядів невід'ємна;
- `chk_sub_no_self : subscriber_id <> channel_id` — заборона самопідписки.

5.5. NOT NULL

Обмеження `NOT NULL` застосовано до всіх полів, які логічно не можуть бути порожніми: ідентифікатори, заголовки, `user_id` у відео, `text` у коментарях, `is_like` тощо.

5.6. DEFAULT

- `user.is_channel` — `DEFAULT FALSE` ;
- `user.created_at` , `video.created_at` , `comment.created_at` , `like.created_at` , `subscription.created_at` — `DEFAULT NOW()` ;
- `video.views` — `DEFAULT 0` ;
- `video.is_public` — `DEFAULT TRUE` .

6. Висновки

У ході виконання лабораторної роботи №2 створено реляційну схему бази даних для відеохостинг-платформи **VideoHub**. Було реалізовано п'ять таблиць, між якими встановлено всі необхідні зв'язки через зовнішні ключі. Застосовано різноманітні механізми забезпечення цілісності даних:

- **PK** для унікальної ідентифікації рядків;
- **FK** з `ON DELETE CASCADE` для підтримки референційної цілісності;
- **UNIQUE** для унікальності нікнеймів, email та комбінацій;
- **CHECK** для контролю допустимих значень (додатна тривалість, невід'ємні перегляди, заборона самопідписки);

- **NOT NULL** та **DEFAULT** для підтримки коректного стану даних.

Таблиці заповнені тестовими даними, які відповідають реальній логіці роботи відеохостингу: канали завантажують відео, користувачі залишають коментарі та ставлять лайки, підписуються один на одного. Також підготовлено приклади закоментованих порушень обмежень, які можна використати для демонстрації роботи СКБД PostgreSQL.